

Проектиране и интегриране на софтуерни системи

Ивайло Андреев + gemini 3 Flash, Claude Sonnet 4.6

22 юни 2026

Съдържание

1	Характеристики на разпределените софтуерни системи	3
1.1	Дефиниции	3
1.2	Основни характеристики на системите	3
1.3	Видове разпределени системи	3
1.3.1	1. Разпределени изчислителни системи (Distributed Computing Systems)	4
1.3.2	2. Разпределени информационни системи (Distributed Information Systems)	4
1.3.3	3. Широко разпространени / Повсеместни разпределени системи (Pervasive Distributed Systems)	4
1.4	Архитектурни тенденции	4
2	Междупроцесна комуникация (Inter-Process Communication - IPC)	5
2.1	Отдалечено извикване	5
2.1.1	1. Remote Procedure Call (RPC)	6
2.1.2	2. Remote Method Invocation (RMI)	6
2.1.3	3. CORBA (Common Object Request Broker Architecture)	8
2.2	Мултикаст (Multicast)	8
3	Разпределени обекти и компоненти	8
3.1	Разпределени обекти	8
3.2	Разпределени компоненти	9
4	Уеб услуги (Web Services)	9
4.1	Дефиниции и архитектурни цели	9
4.2	Шаблони за комуникация (Communication Patterns)	9
4.3	Стандарти за уеб услуги	10
4.3.1	1. SOAP (Simple Object Access Protocol)	11
4.3.2	2. WSDL (Web Service Description Language)	11

4.3.3	3. UDDI (Universal Description, Discovery, and Integration)	11
4.4	Message Brokers (опашки за съобщения)	12
4.5	Сравнение: RMI срещу Уеб услуги (SOAP/WSDL)	12

1 Характеристики на разпределените софтуерни системи

1.1 Дефиниции

В съвременното софтуерно инженерство (по Таненбаум и стандарта IEEE) са утвърдени следните две основни дефиниции за разпределена софтуерна система:

- Група от независими компютри, която за крайния потребител изглежда и се възприема като една единствена, цялостна система.
- Система, в която хардуерни или софтуерни компоненти, намиращи се на мрежови компютри (възли), комуникират помежду си и координират своите действия изцяло и единствено чрез изпращане и обмен на съобщения.

1.2 Основни характеристики на системите

Разпределените системи се проектират с цел постигане на следните пет ключови софтуерни архитектурни качества:

1. **Свързаност и комуникация:** Възможност за безпрепятствен обмен на ресурси и данни между отдалечените възли.
2. **Хетерогенност:** Способност на системата да работи интегрирано, независимо от различията в хардуера, операционните системи, мрежовите протоколи и езиките за програмиране.
3. **Мащабируемост (Scalability):** Капацитет на системата да реагира адекватно при значително нарастване на натоварването. Реализира се по два начина: **хоризонтално мащабиране** (*horizontal scaling*) — добавяне на нови възли към системата, и **вертикално мащабиране** (*vertical scaling*) — увеличаване на ресурсите (CPU, RAM) на съществуващите възли.
4. **Отказоустойчивост при частични откази:** Способност на архитектурата да продължи стабилното си функциониране, дори когато определени възли или комуникационни канали отпадат.
5. **Сигурност и надеждност:** Защита на данните и транзакциите при пренос по публични или споделени мрежи.

1.3 Видове разпределени системи

Разпределените софтуерни системи се класифицират в три основни категории:

1.3.1 1. Разпределени изчислителни системи (Distributed Computing Systems)

Използват се за тежки изчислителни задачи, изискващи изключително висока производителност. Разделят се на:

- **Клъстери (Cluster Computing):** Силно свързани (*tightly coupled*) системи. Възлите са идентични като хардуер и операционна система, физически близо един до друг, свързани в една високоскоростна локална мрежа, като една програма работи паралелно върху всички тях.
- **Гридове (Grid Computing):** Слабо свързани (*loosely coupled*) хетерогенни системи, при които хардуерът, операционните системи и мрежите на отделните възли се различават драстично. Възлите често са географски и физически силно отдалечени един от друг.

1.3.2 2. Разпределени информационни системи (Distributed Information Systems)

Проектирани са с цел интеграция и свързване на множество разнообразни приложения, които си комуникират по мрежата, но първоначално могат да бъдат оперативно несъвместими. Интеграцията се извършва на няколко нива:

- **Клиент-Сървър (Client-Server):** На по-ниско технологично ниво клиентите групират и изпращат заявки към сървъра, които се изпълняват по "ато-марен" начин (в тяхната цялост).
- **Peer-to-Peer (P2P):** На по-високо ниво, където всяко приложение (възел) действа едновременно като клиент и като сървър, позволявайки директна комуникация помежду им без централен посредник.

1.3.3 3. Широко разпространени / Повсеместни разпределени системи (Pervasive Distributed Systems)

За разлика от стабилните промишлени системи, тези системи по дефиниция са изключително нестабилни. Това се налага поради масовото навлизане на мобилни, преносими и вградени (embedded) изчислителни системи.

- Възлите в тези системи са малки, мобилни, захранват се от батерии и използват безжични комуникации.
- Характеризират се с ад-хок свързване и пълна липса на нужда от непрекъснат човешки надзор или администрация. Примери за това са умните коли, авиационните системи и IoT сензорните мрежи.

1.4 Архитектурни тенденции

- Масово навлизане на повсеместната изчислителност и IoT уреди (смарт часовници, коли, сензори).

- Постоянно разширяване на Интернет като глобална разпределена мрежа и драстично нарастване на търсенето на мултимедийни и стрийминг услуги (Webcasting).
- Доминиране на разпределени системи "по заявка" (*on-demand*) и Облачни изчисления (*Cloud Computing* като AWS, Azure, Google Cloud).

2 Междупроцесна комуникация (Inter-Process Communication - IPC)

Комуникацията между процесите в разпределена среда се базира на обмен на съобщения. Ядрото на операционната система предоставя нисконивови механизми за IPC като **опашки от съобщения (message queues)**, **семафори** и **споделена памет (shared memory)**.

В разпределените системи тези IPC механизми рядко се използват директно от програмистите. Вместо това те се капсулират от **междинен софтуер (middleware)**, който предоставя високо ниво на абстракция чрез интерфейси. Middleware скрива хетерогенността на хардуера, операционните системи и мрежата, като осигурява стандартизирана комуникация, прозрачност и координация между разпределените компоненти.

Основни функции на middleware:

- Именуване и адресиране на компоненти
- Маршалинг/демаршалинг на данни (сериализация)
- Управление на транзакции
- Сигурност и автентикация
- Балансиране на натоварването

Важна част от този middleware е *message-passing* механизмът за комуникация без споделена памет.

Комуникацията се разделя на два типа:

- **Connection-oriented:** Изисква изграждането на изричен, специален канал за комуникация между процесите.
- **Message-oriented:** Преизползва вече съществуващи мрежови канали за изпращане на независими пакети данни.

2.1 Отдалечено извикване

Отдалечените извиквания позволяват на процес А да използва функционалност, намираща се в процес В, по напълно "прозрачен" за себе си начин — маскирайки факта, че В е външен процес, намиращ се на друга машина. Имплементира се чрез две основни технологии:

2.1.1 1. Remote Procedure Call (RPC)

RPC позволява извикване на процедура на отдалечен възел, прозрачно за извикващия — като локална функция. Middleware се грижи за сериализацията на параметрите (**маршалинг**) и десериализацията на резултата (**демаршалинг**).

RPC се базира на предоставянето на интерфейси, които специфицират процедурите, предлагани от отдалечения процес, скривайки изцяло имплементацията им. Интерфейсите се описват чрез **IDL (Interface Definition Language)**, чиято цел е да позволи процедура, написана на един език, да бъде извикана от клиент, написан на съвсем различен език. Модерна имплементация на този подход е протоколът **gRPC** на Google (базиран на HTTP/2 и Protocol Buffers).

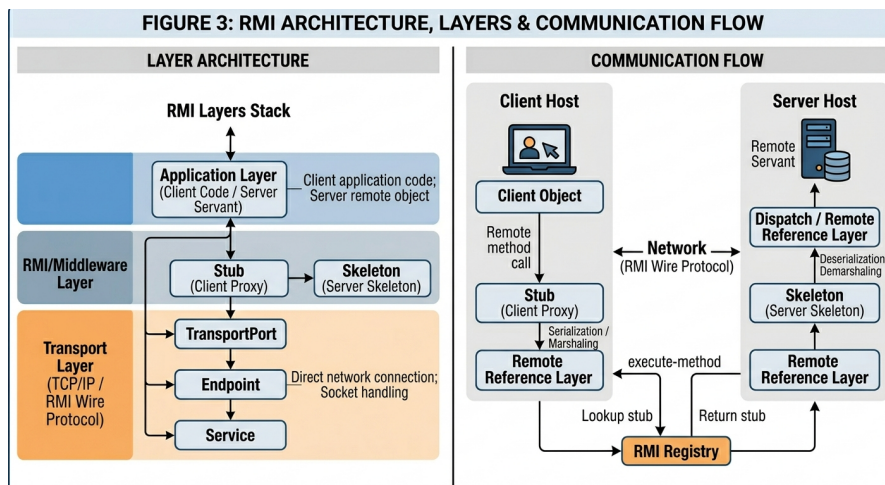
RPC дефинира три основни семантики на извикване при възникване на грешки в мрежата:

- **maybe:** Процедурата се изпълнява веднъж или изобщо не се изпълнява, като клиентът не получава потвърждение.
- **at-least-once:** Клиентът гарантирано получава резултат само ако процедурата се е изпълнила поне веднъж (възможни са повторения при препредаване на заявката).
- **at-most-once:** Клиентът знае, че процедурата се е изпълнила най-много веднъж, елиминирайки риска от дублиране на операции.
- **exactly-once:** Теоретично гарантира точно едно изпълнение, но е изключително трудно постижимо на практика в ненадеждни мрежи, тъй като изисква синхронизация между изпращача и получателя.

2.1.2 2. Remote Method Invocation (RMI)

RMI е обектно-ориентираният аналог на RPC за Java. Позволява извикване на методи на отдалечени обекти. Използва **stub** (клиентска страна) и **skeleton** (сървърна страна) за прозрачна комуникация.

- **Прилики с RPC:** Поддържа строг модел на програмиране с интерфейси, базира се на нисконивовия протокол заявка-отговор и осигурява високо ниво на прозрачност.
- **Разлики от RPC:** Поддържа изцяло обектно-ориентираната парадигма и позволява предаването на реални референции към обекти като параметри на методите.



Фигура 1: Архитектурен модел, слоеве и динамичен поток на RMI комуникация

На Фиг. 1 са илюстрирани два взаимно свързани аспекта на технологията: **слоевата архитектура** (вляво) и **динамичният комуникационен поток** (вдясно). Ключовият архитектурен елемент в динамичния модел е **RMI Registry** — разпределена директорийна услуга (наименуващ регистър), работеща на специфичен порт на сървъра. Ролята му е да служи като посредник при първоначалното свързване. Процесът протича на три етапа:

1. **Регистрация (Bind):** Сървърният обект (Remote Servant) се регистрира в RMI Registry под уникално текстово име.
2. **Търсене (Lookup):** Клиентът изпраща заявка към RMI Registry по името на услугата и получава обратно локално прокси референция (*Stub*).
3. **Директна комуникация:** След като клиентът притежава *Stub*-а, RMI Registry излиза от комуникационния канал. Извикването на метод се маршализира от Remote Reference слоя на клиента, пренася се по мрежата чрез специфичния RMI Wire протокол и се демаршализира от слоевете на сървъра (Skeleton), задействайки реалния Servant обект.

Имплементационната архитектура на RMI се състои от следните задължителни модули (виж Фиг. 1):

- **Клиент (Client Application):** Инициира извикването на метода.
- **RMI Слой:** Съдържа Client Proxy (Stub), Servant Skeleton, Remote Reference Module (за маршализация на данните) и Dispatcher.
- **Маршализация/Демаршализация:** Процесите на сериализация/десериализация на обектите при предаване по мрежата.

- **Сървър (Servant/Dispatcher):** Реалната инстанция на отдалечения обект.
- **Транспортен слой:** Управлява физическата TCP/IP връзка по мрежата.

2.1.3 3. CORBA (Common Object Request Broker Architecture)

CORBA е платформено-независим стандарт на OMG за комуникация между разпределени обекти. Той решава хетерогенността, като позволява на компоненти, написани на различни езици (C++, Java, Python), да комуникират прозрачно чрез IDL-дефинирани интерфейси.

Ключови компоненти на CORBA:

- **ORB (Object Request Broker):** Посредникът, който маршрутизира заявките между обектите.
- **IDL (Interface Definition Language):** Езиково-неутрален език за описание на интерфейсите.
- **IIOP (Internet Inter-ORB Protocol):** Протоколът за комуникация между ORB-овете.
- **Naming Service:** Регистър за именуване на обекти (аналог на DNS).

2.2 Мултикаст (Multicast)

Мултикастът е метод за едновременно изпращане на едно съобщение от един изпращач до дефинирана група от много получатели в мрежата. Мрежовата топология на тези групи се оформя като **дърво** (директен достъп) или **mesh** (наличие на множество алтернативни пътища).

Тъй като рутерите на приложно ниво не се борят за междинни възли, рутването се оптимизира чрез епидемични алгоритми — **Gossip-Based Data Dissemination**. Подобно на биологичен вирус, всеки възел, който притежава информацията ("заразен" възел), я предава автономно на своите съседни възли. Изтриването на информация в такива децентрализирани епидемични мрежи е трудно, поради което се симулира "изтриване чрез модификация" (death certificates).

3 Разпределени обекти и компоненти

3.1 Разпределени обекти

При този дизайн архитектурните единици са разпределени обекти, които комуникират чрез RMI или механизма на разпределените събития. Те осигуряват капсулация и силна абстракция на данните.

Основни дефекти и проблеми на обектния подход:

- Интерфейсите на обектите описват само методите, но не дефинират изрично контекстните зависимости на имплементацията им.

- Програмистите са принудени ръчно да имплементират и управляват сложни системни детайли като конкурентност, нисконивова сигурност и обработка на мрежови грешки.
- Липсва автоматизирана (*out-of-the-box*) поддръжка за управление на жизнения цикъл и сложни конфигурации от обекти.

3.2 Разпределени компоненти

Проблемите на обектите налагат прехода към парадигмата на разпределените компоненти. **Софтуерният компонент** е независима градивна единица, която задължително дефинира както интерфейсите, които предоставя (*provided interfaces*), така и своите **експлицитни контекстни зависимости** (*required interfaces*). Инфраструктурата на компонентите се управлява автоматизирано от контейнери, които поемат сигурността и конкурентността. Индустриални стандарти за разпределени компоненти са **Enterprise Java Beans (EJB)** и архитектурата **CORBA**.

4 Уеб услуги (Web Services)

4.1 Дефиниции и архитектурни цели

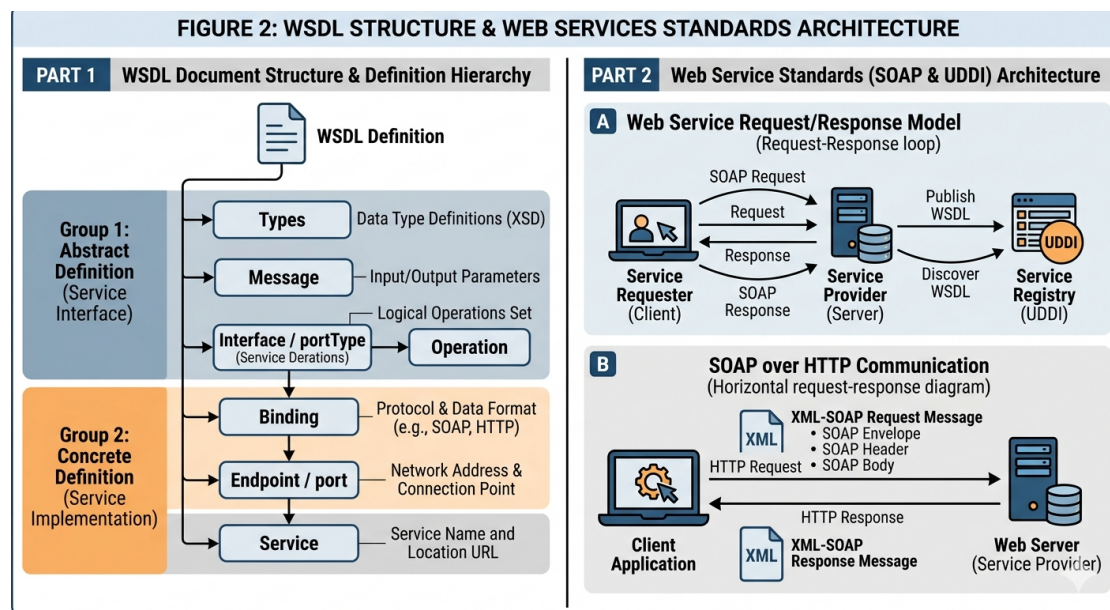
- **Интерфейс на уеб услуга:** Колекция от операции, изложени и достъпни по стандартизиран начин през Интернет, които се имплементират от хетерогенни ресурси (програми, бази данни).
- **Транспорт:** Уеб услугите се достъпват предимно чрез приложния протокол **HTTP**.
- **Цел:** Основната цел е пълно абстрахиране и изолиране на разпределеното компютърно изпълнение от специфичните детайли на имплементацията (езици за програмиране, платформи). Стандартно се изграждат чрез XML-SOAP.

4.2 Шаблони за комуникация (Communication Patterns)

Различават се три основни модела за обмен на съобщения:

1. **Синхронна комуникация:** Клиентът изпраща заявка и преминава в блокиращо състояние, очаквайки пристигането на отговора от сървъра.
2. **Асинхронна комуникация:** Клиентът изпраща съобщението и продължава изпълнението си, без да блокира до изчакване на отговора.
3. **Комуникация, базирана на събития:** Обмен на съобщения, иницииран автоматично при настъпване на специфично събитие в системата (напр. *webhooks*).

4.3 Стандарти за уеб услуги



Фигура 2: Структура на техническото WSDL описание (вляво) и архитектурна топология на SOAP/UDDI интеграцията (вдясно)

На Фиг. 2 подробно са декомпозирани стандартите за уеб услуги:

- **Вляво (PART 1):** Йерархичната структура на WSDL документа. Тя строго разделя уеб услугата на **абстрактна част** (дефинираща логическия интерфейс чрез *Types*, *Message* и набора от операции — *Interface / portType*) и **конкретна част** (описваща физическата имплементация чрез транспортния протокол *Binding*, мрежовия адрес *Endpoint / port* и крайната точка *Service*).
- **Вдясно (PART 2):** Интеграционната топология и съобщителният модел.

Секция **PART 2 (A)** илюстрира класическия трикомпонентен модел (Service Triangle). Важно е да се отбележи логическият поток на откриване на услуги: доставчикът (*Service Provider*) публикува техническото WSDL описание в споделения UDDI регистър (*Publish WSDL*). От своя страна, клиентът (*Service Requester*) се обръща към UDDI регистъра, за да открие и сваля съответния WSDL контракт (*Discover WSDL*), след което инициира директна комуникация със сървъра.

В секция **PART 2 (B)** е показан самият транспорт на съобщенията. Комуникацията се осъществява по модела заявка-отговор (*Request-Response loop*) над приложния протокол HTTP. Данните се капсулират в строго дефинирани XML-SOAP съобщения, чиято структура задължително съдържа коренов елемент *SOAP Envelope*, метаданни в *SOAP Header* и приложен товар в *SOAP Body*.

4.3.1 1. SOAP (Simple Object Access Protocol)

SOAP е протокол за съобщения, имплементиран над HTTP или SMTP, целящ размяна на структурирана информация под формата на **XML обекти**. SOAP комуникацията (виж Фиг. 2, вдясно) винаги се осъществява чрез HTTP заявки и отговори.

SOAP дефинира три задължителни елемента в своята спецификация:

- **Envelope (Плик):** Кореновият XML елемент, който дефинира документа като SOAP съобщение.
- **Header (Хедър):** Опционален елемент, съдържащ метаданни (сигурност, маршрутизация, транзакции).
- **Body (Тяло):** Задължителен елемент, съдържащ реалните данни на заявката или отговора.

SOAP поддържа строго типизиран контракт, транзакции (WS-AtomicTransaction) и сигурност (WS-Security), което го прави подходящ за enterprise среди.

4.3.2 2. WSDL (Web Service Description Language)

WSDL е базиран на XML технически език, който специфицира интерфейса на дадена уеб услуга и детайлите за комуникация с нея. Той служи като допълнение към SOAP. Структурата на WSDL документа (Фиг. 2, вляво) е разделена на:

- **Абстрактна част (Abstract Definition):** Дефинира логическите интерфейси. Те включват типовете данни (*Types*), структурата на съобщенията (*Message*) и предлаганите операции (*Interfaces / portTypes*).
- **Конкретна част (Concrete Definition):** Специфицира физическата имплементация. Тя описва обвързването към транспортен протокол (*Bindings*), точната крайна точка (*Endpoints / ports*) и конкретното мрежово местоположение на услугата (*Services*).

4.3.3 3. UDDI (Universal Description, Discovery, and Integration)

UDDI е разпределена директорийна услуга (регистър), действаща като "телефонен указател" за бизнеса. Тя позволява на организациите да публикуват своите уеб услуги (описани чрез WSDL), а на клиентите — да ги откриват динамично.

UDDI архитектурно се състои от три логически компонента:

- **Бели страници (White Pages):** Съдържат базова бизнес информация за доставчика на услугата (име, адрес, контакт).
- **Жълти страници (Yellow Pages):** Използват се за индустриална и стандартизирана класификация на предлаганите услуги.

- **Зелени страници (Green Pages):** Съдържат детайлната техническа и имплементационна информация за уеб услугите и начините за интеграция с тях.

4.4 Message Brokers (опашки за съобщения)

Message broker-ите осигуряват **асинхронен** обмен на съобщения между компонентите на разпределените системи. Издателят изпраща съобщение до брокера, а консуматорът го получава когато е готов — двата компонента не трябва да са активни едновременно.

Основни функции на message broker-ите:

- **Буфериране:** Съобщенията се съхраняват до тяхната обработка.
- **Маршрутизиране:** По тематичен принцип (topic-based) или базирано на съдържанието (content-based).
- **Гарантирана доставка:** Поддържат се семантики като exactly-once и at-least-once.
- **Мащабируемост:** Разпределени опашки при голям товар.

Примери за message broker-и:

- **Apache Kafka:** Log-базиран, с висока производителност и пропускателна способност.
- **RabbitMQ:** Базиран на AMQP протокола.
- **Apache ActiveMQ** и **AWS SQS**.

4.5 Сравнение: RMI срещу Уеб услуги (SOAP/WSDL)

Прилики: И двата модела се базират на протокола заявка-отговор (*Request-Response*), целят пълна прозрачност за клиента и прилагат строго разделение на слоеве, за да скрият детайлите по мрежовия пренос от разработчика.

Разлики:

- **Технологична обвързаност:** RMI е силно специализирано решение (обикновено за Java), докато SOAP/WSDL са напълно хетерогенни — независими от езика и платформата.
- **Откриване на услуги:** RMI разчита на лек RMI Registry, докато уеб услугите ползват мащабната UDDI директория (бели, жълти и зелени страници).
- **Формат на данните:** RMI използва двоична сериализация на Java обекти, докато SOAP предава структуриран XML, капсулиран в HTTP/SMTP плик.